JOB SHOP PROBLEM (JSP)
GUÍA PRACTICA

Leslie Pérez Cáceres

¿Qué aprenderemos?

En este documento, aprenderemos sobre uno de los problemas clásicos en la Investigación de operaciones, conocido como Job Shop Problem (JSP), o Job Shop Scheduling Problem (JSSP). Además veremos distintas aplicaciones, el modelo matemático y técnicas de resolución

---

Descripción general

En la industria de manufactura, generalmente los productos se fabrican secuencialmente en una cadena de procesos, lo que genera un gran desafío al buscar ordenar esta secuencia de tareas, para así poder fabricar una gran cantidad de productos de calidad en un tiempo reducido. Para esto, se debe considerar el tamaño de las instalaciones, la maquinaria disponible, la cantidad de tareas necesarias, materias primas, entre otros aspectos.

El problema que se genera al ordenar la secuencia de tareas para fabricar un determinado producto recibe el nombre de Job Shop Problem, un problema de alta complejidad, pero con un valor de aplicación muy grande. El JSP básicamente busca ordenar las distintas tareas (o trabajos) necesarias para fabricar un producto, considerando ciertas restricciones y parámetros para obtener la mejor secuencia de tareas, y saber los tiempos de inicio y fin para la producción. Por ejemplo, supongamos que tenemos una situación donde en una planta, procesamos 3 componentes electrónicos y tenemos tres máquinas disponibles, donde una cada componente se procesa a través de tres operaciones, y una posible configuración entre componentes, sus operaciones y las máquinas es la siguiente:

Trabajos	Operación 1	Operación 2	Operación 3
Componente 1	M_1,3	M_2,2	M_3,4
Componente 2	M_2,5	M_1,1	M_3,2
Componente 3	M_3,3	M_1,4	M_1,2

En base a estos datos, la distribución de las operaciones de cada componente en cada máquina, considerando su orden y tiempo de duración, y partiendo desde la hora 0, sería de la siguiente forma:

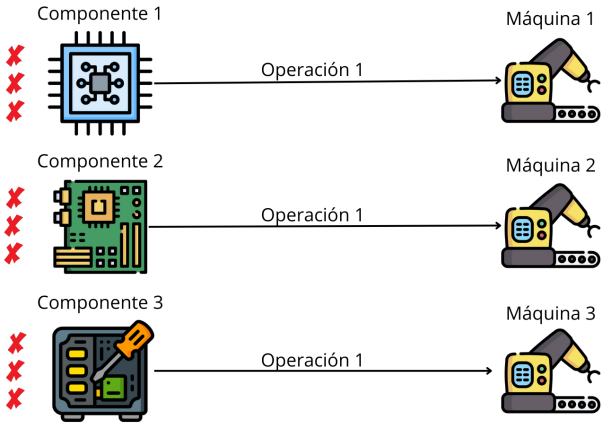


Figura 1: Horas 0-3

Luego de la hora 3, las máquinas 1 y 2 habrán terminado las operaciones 1 de los componentes 1 y 2 respectivamente lo cual se visibiliza de mejor forma al costado de cada componente, donde la maquina 2 seguirá trabajando con el componente 2.

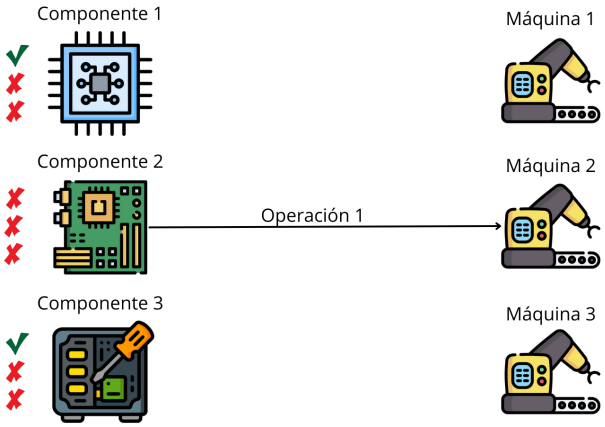


Figura 2: Hora 3

Después, el componente 3 ocupara la máquina 1 para continuar con su operación 2 mientras el componente 1 quedara en espera de la liberación de la maquina 2 para realizar su operación 2.

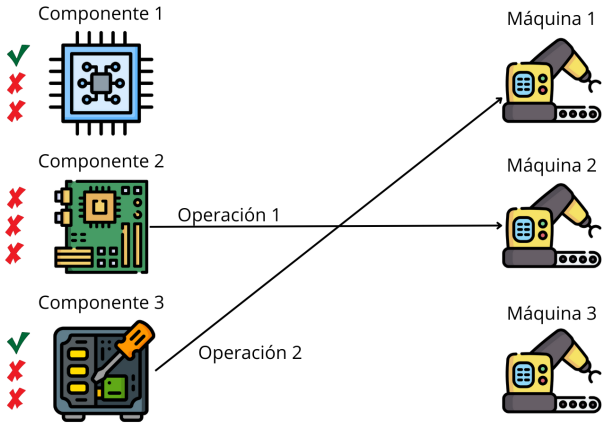


Figura 3: Horas 3-5

Cuando la maquina 2 termine con el componente 2 dentro de la hora 5, el componente 1 ahora entrara en esta maquina para realizar su operación 2.

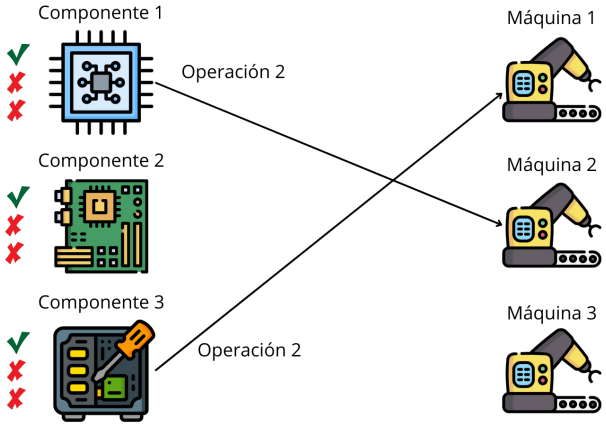


Figura 4: Horas 5-7

Una vez llegada la hora 7, las máquina 1 y 2 quedaran libres junto a la máquina 3, por lo cual se asignaran los componentes 1, 2 y 3 a las máquinas 3 , 1 y 2 respectivamente para realizar la operación 3 del componente 1, la operación 2 del componente 2 y la operación 3 del componente 3.

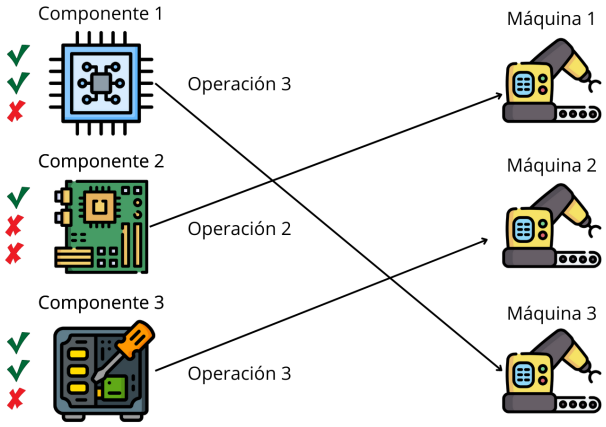


Figura 5: Horas 7-8

La operación 2 del componente 2 finalizara a la hora 8 y esperará a que la máquina 3 quede libre.

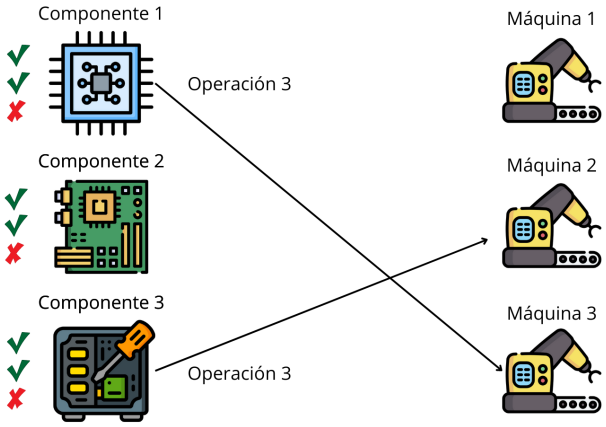


Figura 6: Horas 8-9

En la hora 9 el componente 3 estará completo en su totalidad mientras el componente 1 seguirá en la máquina 3.

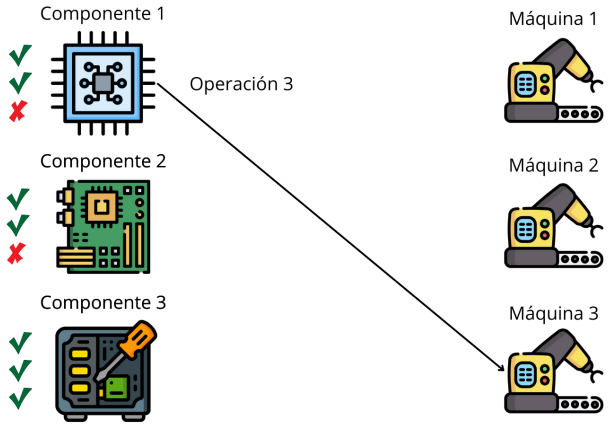


Figura 7: Horas 9-11

El componente 1 estará listo en su totalidad a la hora 11, con lo cual el componente 2 comenzará su operación 3 en la máquina 3 liberada por este componente 1.

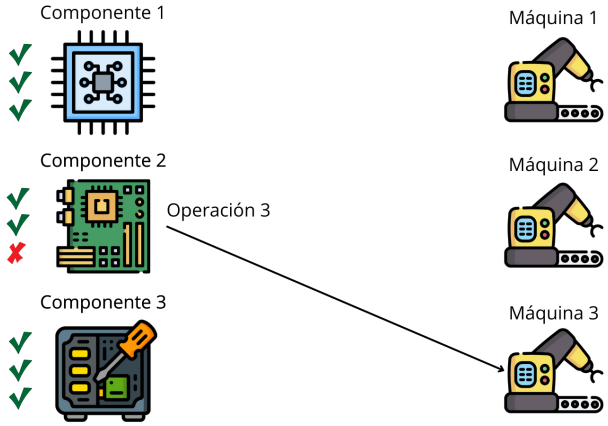


Figura 8: Horas 11-13

Con lo que finalmente a la hora 13 todos los componentes estarán completos y la iteración habrá terminado.

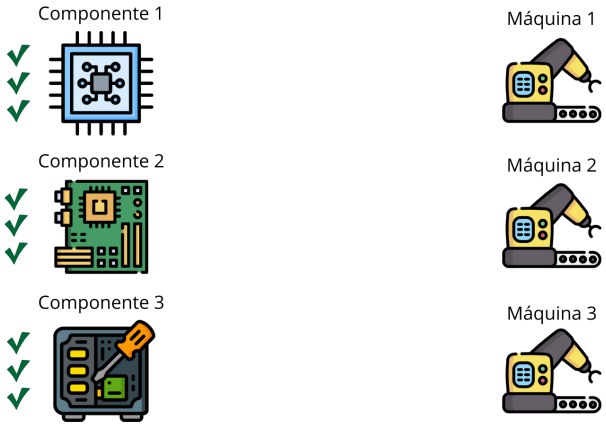


Figura 9: Hora 13

Variantes

1. Flexible Job Shop Problem (FJSP): Es una extensión del JSP clásico, donde cada operación puede ser procesada en más de una máquina. Aquí, el desafío adicional es asignar cada operación a una máquina específica, además de secuenciar las operaciones.
2. Open Shop Problem (OSP): No existe una secuencia predeterminada para las operaciones de cada trabajo. Las operaciones pueden realizarse en cualquier orden, pero aún se deben asignar y secuenciar en las máquinas.
3. Flow Shop Problem (FSP): Todas las operaciones deben seguir la misma secuencia de máquinas. Es una versión simplificada del JSP, ya que la secuenciación es idéntica para todos los trabajos.
4. No-Wait Job Shop Problem (NWJSP): Una variante donde no se permite que un trabajo espere entre operaciones. Una vez que un trabajo comienza, debe ser procesado en todas las máquinas sin interrupciones.
5. Job Shop Scheduling with Sequence-Dependent Setup Times (JSP-SDST): Aquí, el tiempo de configuración (setup time) entre operaciones depende de la secuencia en la que las operaciones se realizan en la máquina.

Contexto de Uso

El problema de Job Shop (JSP) se aplica en diversos contextos, desde la manufactura industrial hasta la gestión de servicios y actividades cotidianas. En la industria, se utiliza para optimizar la secuenciación de operaciones en procesos de fabricación, como en la producción de automóviles, componentes electrónicos, y productos metálicos. También se aplica en la planificación de recursos en hospitales, logística, distribución, y desarrollo de software, donde la asignación eficiente de recursos limitados es crucial para minimizar tiempos y costos. En la vida diaria, el JSP se refleja en la gestión del tiempo personal, la planificación de eventos, y la organización de actividades familiares, donde la optimización de tareas y recursos mejora la eficiencia y la coordinación. Estos ejemplos muestran cómo los principios del JSP son fundamentales tanto en entornos complejos como en situaciones cotidianas.

Modelo

El Job Shop Problem corresponde a un problema NP-hard*, el cual está definido por un conjunto de trabajos que se deben ejecutar en un conjunto de máquinas, con un orden específico para cada trabajo. Cada trabajo j perteneciente a un conjunto J tiene un tiempo de ejecución establecido para cada maquina m perteneciente a un conjunto M, y una secuencia específica de operaciones. Cada trabajo debe ser ejecutado en solo una máquina a la vez, y cada máquina debe ejecutar un solo trabajo al mismo tiempo. Además, una vez que se inicia la ejecución de un trabajo, no puede detenerse hasta su finalización. El objetivo del problema es minimizar el makespan (tiempo máximo de finalización) de todos los trabajos. Para definir el modelo matemático, primero definiremos algunas variables y parámetros.



Recordar que un problema NP-hard es aquel que, para instancias muy grandes, el costo computacional de resolverlo es enorme, y no se logran resolver en un tiempo razonable utilizando técnicas clásicas, y no existe método que sea efectivo para todas las instancias del problema.

Parámetros:

- J: Conjunto de todos los trabajos, $J = \{1,2,...,n\}$.
- M: Conjunto de todas las máquinas disponibles, $M = \{1,2,...,m\}$.
- o_{jr}: Máquina que procesa la operación r perteneciente al trabajo j, parte del conjunto $O_j = \{o_{j1},o_{j2},o_{j3},...,o_{jm}\}$, que representa la secuencia de operaciones.
- p_{ij}: tiempo de ejecución de la operación j en la máquina i.
- A: Constante con un valor lo suficientemente grande para corregir las restricciones. Una forma de estimar este valor puede ser la suma de todos los tiempos de procesamiento.

Variables de decisión:

- x_{ij}: tiempo de ejecución del trabajo $j \in J$ en la máquina $i \in M$.
- y_{ijk}: Variable binaria que indica si el trabajo j precede al trabajo k en la máquina i, $i \in M$; $j,k \in J$; $j \neq k$
- C: Variable que representa el makespan.

Ahora, definimos el modelo:

$$min\ Z = max\ C \tag{1}$$

Tenemos la restricción de procedencia, donde aseguramos que en cierta máquina un trabajo comience solo cuando concluya el trabajo en la máquina anterior.

$$x_{o_{jr}j} \geq x_{o_{j(r-1)}j} + p_{o_{jr}j}, \forall r \in 2,...,m, j \in J \tag{2}$$

La siguiente restricción asegura que una máquina esté ejecutando un trabajo solamente.

$$x_{ij} \geq x_{jk} + p_{jk} - Ay_{ijk}, \forall j,k \in J, j \neq k \tag{3}$$

$$x_{ij} \geq x_{jk} + p_{jk} - A(1 - y_{ijk}), \forall j, k \in J, j \neq k \tag{4}$$

Luego, una restricción para calcular correctamente el makespan.

$$C \geq x_{o_{jrj}} + p_{o_{jrj}}, \forall j \in J \tag{5}$$

Por último, las restricciones de dominio para las variables de decisión.

$$x_{ij} \geq 0, \forall j \in J, i \in M \tag{6}$$

$$y_{ijk} \in 0, 1, \forall j, k \in J, i \in M \tag{7}$$

$$C \geq 0 \tag{8}$$

Instancias

Existen diversas formas de representar las instancias del JSP. Una de ellas es el formato de Taillard, que veremos a continuación.

Para una instancia de 3 trabajos y 3 máquinas, tenemos la siguiente tabla:

Trabajos	Operación 1	Operación 2	Operación 3
J_1	M_1,3	M_2,2	M_3,4
J_2	M_2,5	M_1,1	M_3,2
J_3	M_3,3	M_1,4	M_1,2

Cuadro 1: Instancia de 3 trabajos y 3 máquinas

Para el caso anterior, la instancia se representa de la siguiente forma:

Representación de Taillard

```
3 3

3 2 4
5 1 2
3 4 2

1 2 3
2 1 3
3 1 2
```

Lo anterior se describe como:

- La primera línea indica que son 3 trabajo y 3 máquinas.
- Las siguientes 3 filas indican los tiempos de ejecución para cada operación, de cada trabajo, siendo cada fila un trabajo, y cada columna una operación.
- Las ultimas 3 filas corresponden a las máquinas donde se ejecuta cada operación, siendo cada fila un trabajo, y cada columna una operación.

Entonces, para el trabajo J_1, las operaciones se muestran de la siguiente forma:

- Operación 1 en la Máquina 1, con una duración de 3.

- Operación 2 en la Máquina 2, con una duración de 2.
- Operación 3 en la Máquina 3, con una duración de 4.

Esto se repite según las distintas configuraciones entre máquina, operación y duración, por lo que dentro de un archivo de instancia, encontramos varias veces esta estructura.



Pueden encontrar varias instancias para el Job Shop Problem bajo la representación de Taillard en el siguiente link: http://mistic.heig-vd.ch/taillard/problemes.dir/ordonnancement.dir/ordonnancement.html

Otra representación utilizada en la literatura es la de Carlier, la cual está descrita el siguiente artículo : "The one-machine sequencing problem", Jacques Carlier, doi.org/10.1016/S0377-2217(82)80007-6.



Como un pequeño desafío, averigüen sobre otras representaciones para las instancias del JSP.

Métodos de Resolución

1. **Genetic Algorithm:** El algoritmo genético es un método de búsqueda inspirado en los principios de la evolución natural, como la selección, cruce (crossover), y mutación. En el contexto del JSP, las soluciones potenciales (cromosomas) se representan como secuencias de operaciones en las máquinas.
2. **Simulated Annealing:** Es un método inspirado en el proceso físico de enfriamiento de metales, donde se permite que el sistema explore estados de mayor energía (peores soluciones) al principio y, gradualmente, se enfoca en estados de menor energía (mejores soluciones) a medida que el enfriamiento avanza.
3. **Ant Colony Optimization:** El algoritmo de colonia de hormigas es una técnica de optimización basada en el comportamiento colectivo de las hormigas al buscar alimentos, donde las hormigas dejan un rastro de feromonas que guía a otras hormigas hacia soluciones de alta calidad.
4. **Métodos Metaheurísticos Exactos:** Los métodos exactos para resolver el JSP, como la programación entera mixta (MIP) o branch and bound, buscan encontrar la solución óptima explorando exhaustivamente el espacio de soluciones. Sin embargo, debido a la complejidad computacional, estos métodos suelen ser impracticables para instancias grandes del problema.
    - **Programación Entera Mixta (MIP):** Se formula el JSP como un modelo matemático con variables enteras y continuas. Las variables de decisión pueden representar el tiempo de inicio de las operaciones, y las restricciones aseguran que las operaciones respeten el orden de precedencia y la disponibilidad de las máquinas. Los solvers de MIP exploran sistemáticamente el espacio de soluciones utilizando técnicas como corte de planos y ramificación y acotación.
    - **Branch and Bound:** Este método divide el problema en subproblemas más pequeños y utiliza acotaciones para descartar subproblemas que no pueden contener soluciones óptimas, explorando así solo las ramas más prometedoras.



Les proponemos un desafío, el cual consiste en generar y resolver el modelo JSP para el caso presentado a continuación, utilizando la técnica que más les convenga.

Caso de estudio

Una empresa que fabrica distintos tipos de muebles abrirá una nueva planta en Valparaíso, y, debido a la alta demanda, necesita comenzar a producir de forma eficiente lo antes posible. Para esto, buscan organizar las tareas de tal manera que puedan reducir al máximo posible el tiempo de producción, y poder satisfacer a sus clientes. La empresa planea producir 5 tipos de muebles distintos, y dispone de 3 máquinas en la planta. Por lo cual se debe planificar la producción considerando esta situación, y además, tener en cuenta que las operaciones para terminar un mueble tienen un orden específico inalterable y una operación no puede ser interrumpida una vez comenzada. Las operaciones son las siguientes: Ensamblaje, Pintura y Embalaje. Los tiempos de cada operación (en minutos) en cada máquina se muestran en las siguientes tabla:

Trabajos	Ensamblaje	Pintura	Embalaje
Mueble 1	M_1,30	M_2,15	M_3,20
Mueble 2	M_2,25	M_1,10	M_3,10
Mueble 3	M_3,35	M_1,40	M_2,25
Mueble 4	M_2,10	M_3,10	M_1,25
Mueble 5	M_1,20	M_3,25	M_2,20

Cuadro 2: Configuración 1

Trabajos	Ensamblaje	Pintura	Embalaje
Mueble 1	M_3,40	M_1,12	M_2,27
Mueble 2	M_3,15	M_2,15	M_1,30
Mueble 3	M_2,20	M_3,45	M_1,10
Mueble 4	M_1,30	M_2,15	M_3,12
Mueble 5	M_2,25	M_1,20	M_3,20

Cuadro 3: Configuración 2

Trabajos	Ensamblaje	Pintura	Embalaje
Mueble 1	M_2,25	M_3,35	M_1,20
Mueble 2	M_1,20	M_3,20	M_2,20
Mueble 3	M_2,40	M_3,30	M_1,15
Mueble 4	M_3,20	M_1,35	M_2,15
Mueble 5	M_3,10	M_3,35	M_1,15

Cuadro 4: Configuración 3

Encontrar la planificación que minimice el tiempo máximo de producción.